

POLICY FIREWALL

In questo esercizio andiamo a creare una policy firewall che permette alla macchina Kali Linux di effettuare un ping verso la macchina Windows 10 Pro, appartenente alla stessa rete locale precedentemente creata su VirtualBox.

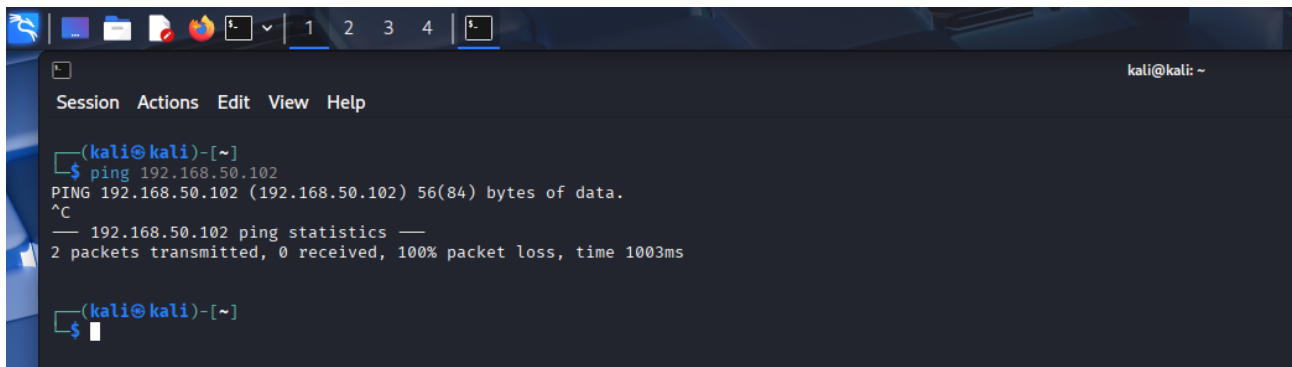
Andiamo nel dettaglio:

dapprima ho verificato che il ping non funzionasse da Kali Linux a Windows (Fig. 1). Una volta accertatomi di ciò, mi sono spostato su Windows ed ho aperto il Firewall di Windows con sicurezza avanzata per impostare una nuova regola personalizzata (Fig. 2).

Successivamente, ho specificato i programmi su cui la regola dovesse avere effetto (Fig. 3), le porte e i protocolli (Fig. 4) e gli indirizzi IP coinvolti (Fig. 5).

Infine, ho impostato l'azione su "Consenti la connessione" (Fig. 6), applicandola a tutti i profili di rete (pubblico, privato e dominio) (Fig. 7).

Terminata la configurazione, mi sono spostato nuovamente su Kali ed ho inviato dei pacchetti ICMP verso la macchina Windows, constatando la corretta ricezione segnando la buona riuscita dell'esercizio. (Fig. 8).



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ping 192.168.50.102  
PING 192.168.50.102 (192.168.50.102) 56(84) bytes of data.  
^C  
— 192.168.50.102 ping statistics —  
2 packets transmitted, 0 received, 100% packet loss, time 1003ms  
  
(kali@kali)-[~]  
$
```

Figura 1 In questa immagine si nota come Kali non sia capace di inviare pacchetti alla macchina Windows

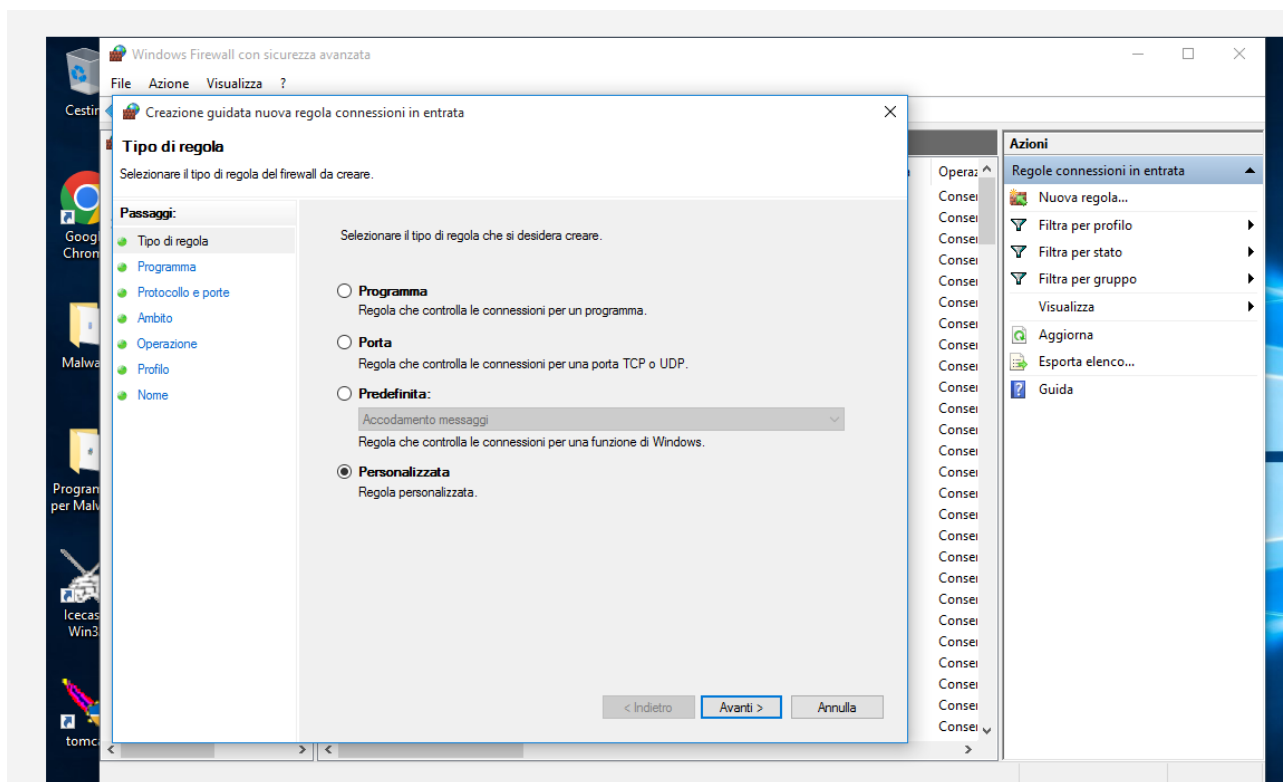


Figura 2 Qui ho iniziato a creare una nuova regola nel firewall di Windows per permettere di ricevere pacchetti da Kali

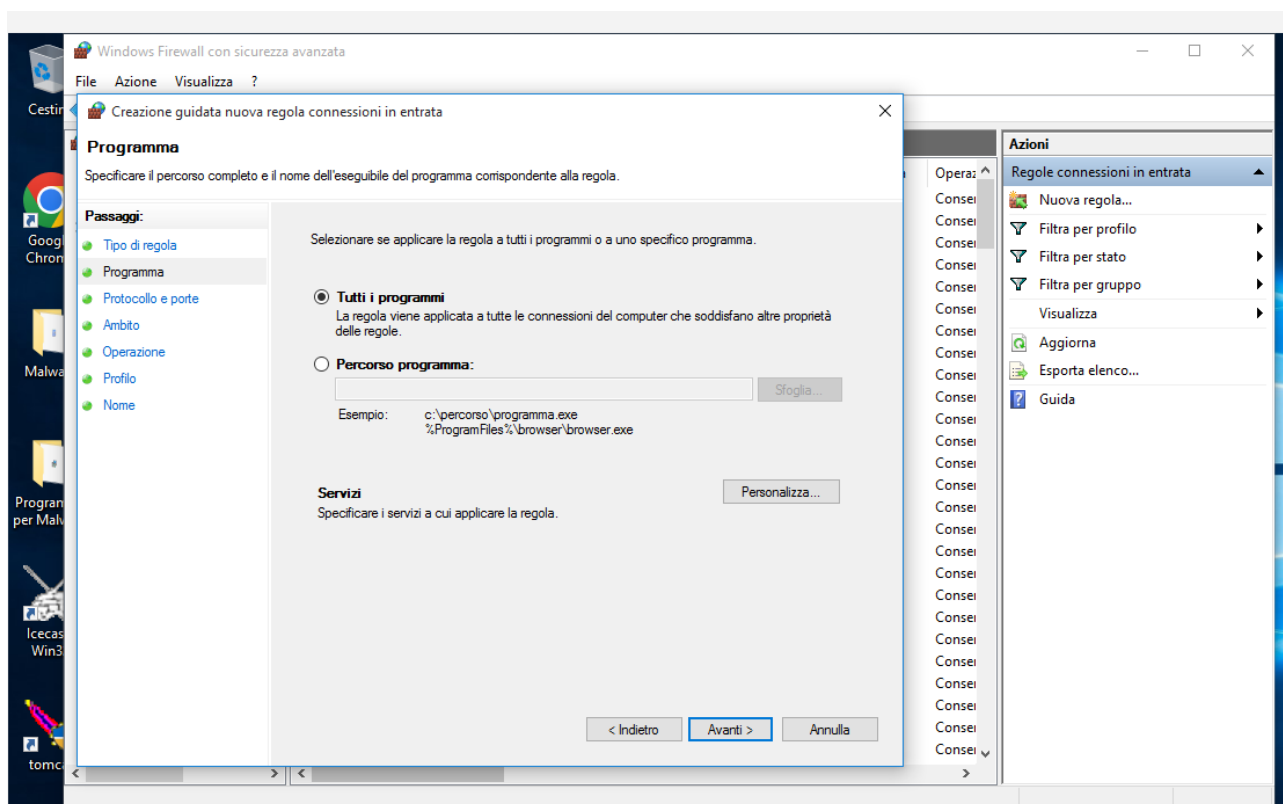


Figura 3 Impostata la regola applicata a tutti i programmi

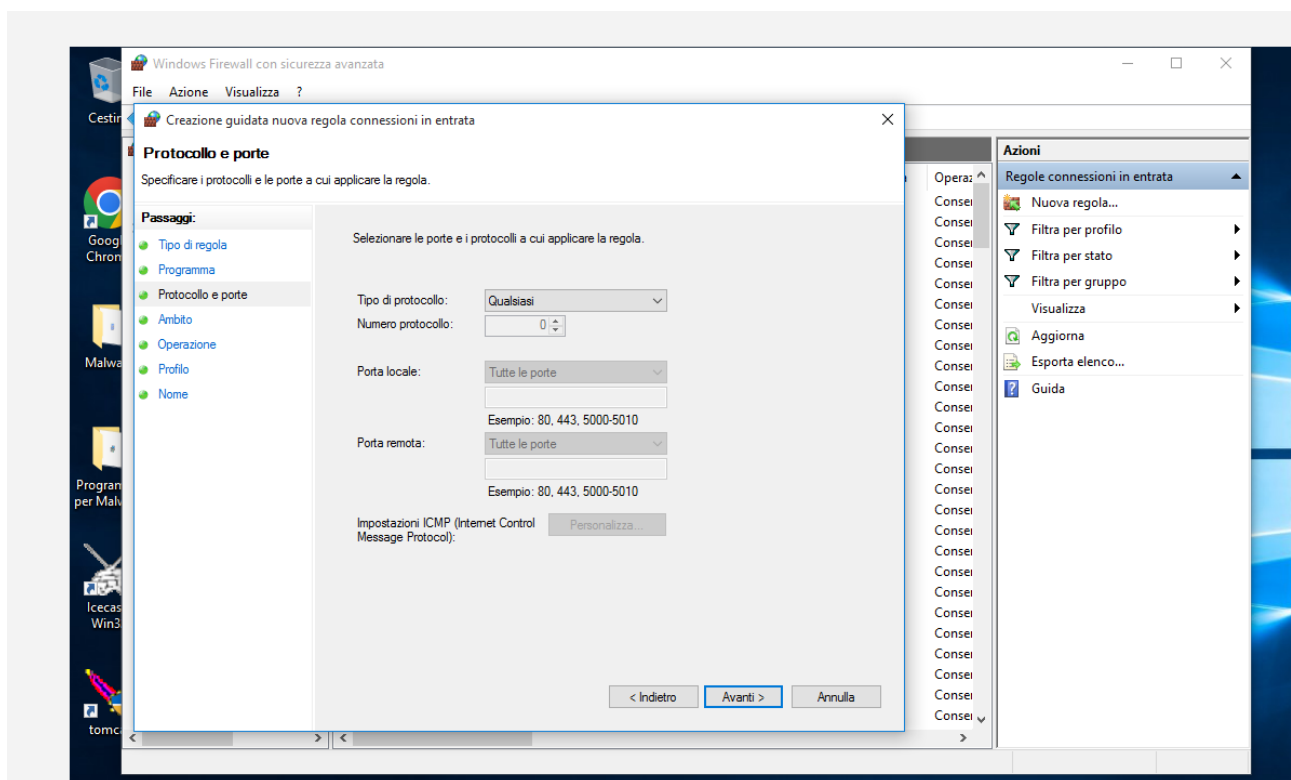


Figura 4 Impostando su qualsiasi tipo di protocollo e su tutte le porte.

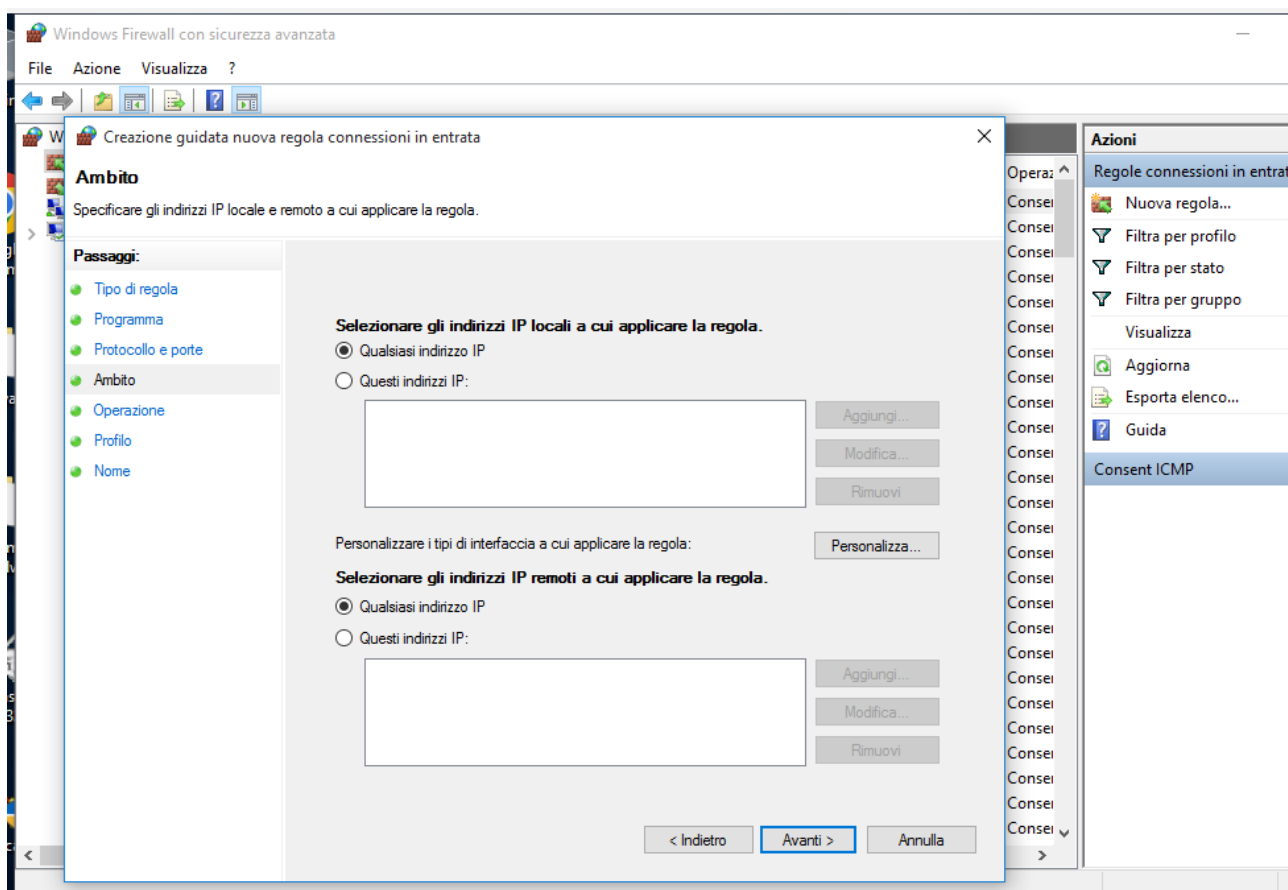


Figura 5 Qui ho selezionato l'opzione "qualsiasi indirizzo IP" così da permettere a Kali e Windows di poter comunicare tra loro.

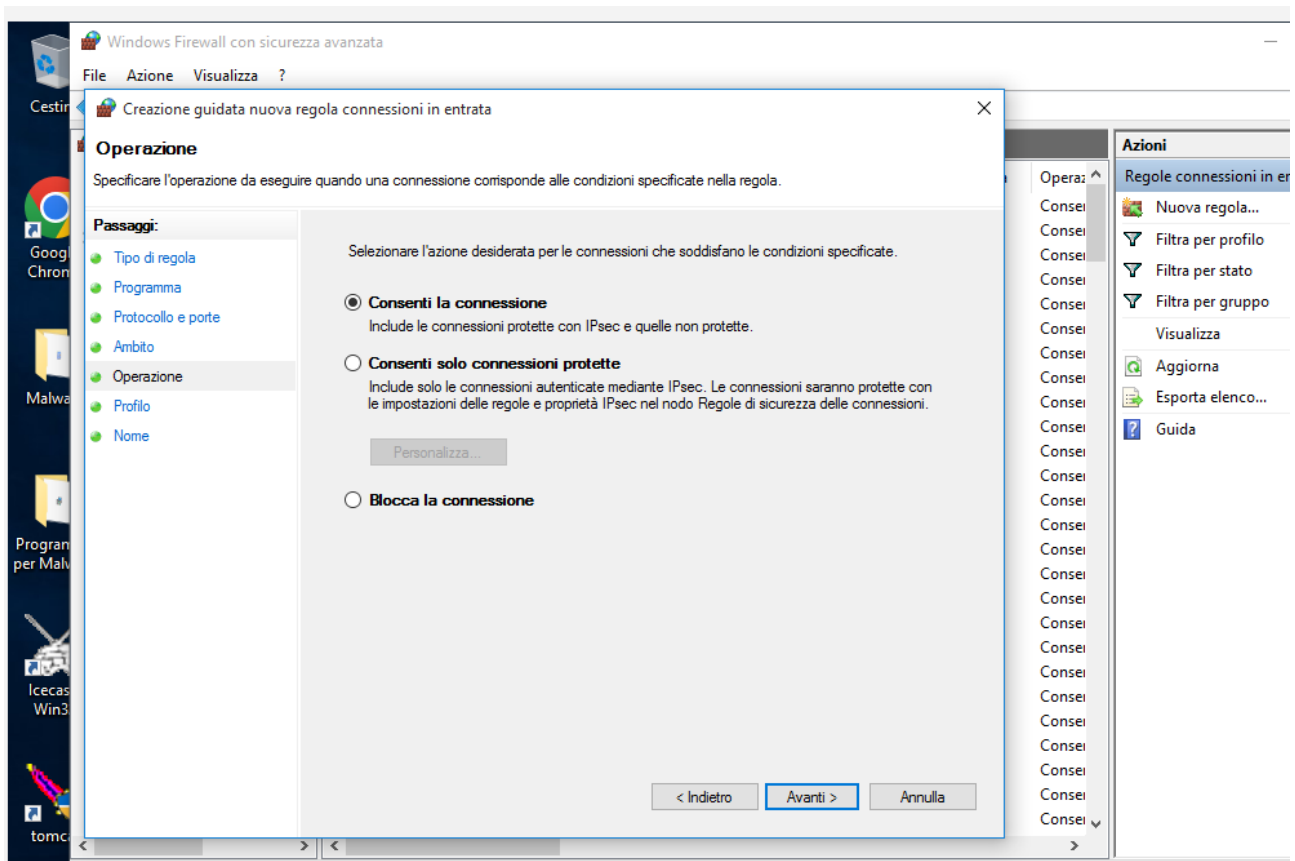


Figura 6 Impostato su Consenti la connessione tra le macchine.

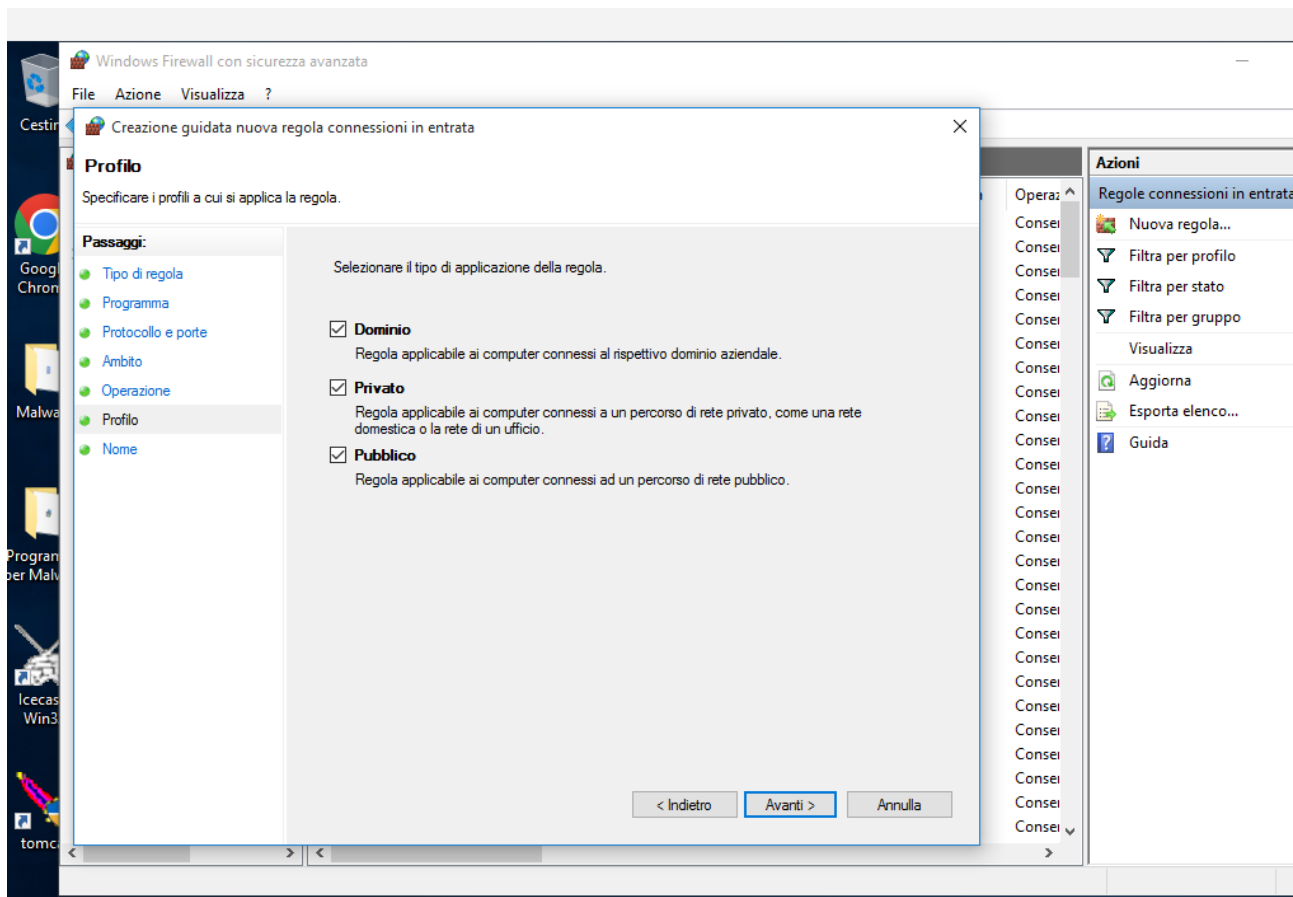


Figura 7 Applicata a tutti i profili.

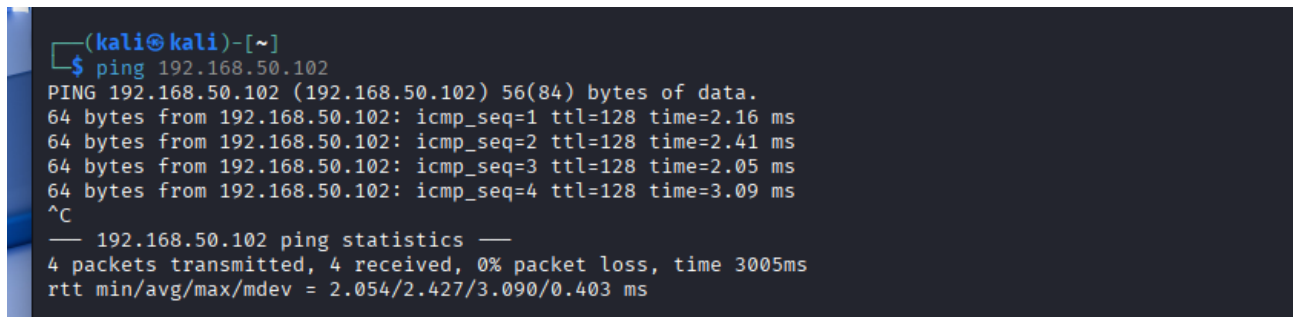


Figura 8 Spostandomi nuovamente su Kali, ho effettuato un ping su Windows, riuscendo finalmente ad inviare pacchetti correttamente.

Venezia, 09/10/2025

Fabio Renna

INETSIM & WIRESHARK

Questo esercizio utilizza il comando Inetsim per emulare il traffico Internet e permette poi di analizzarne i pacchetti con Wireshark.

In primo luogo, su Kali Linux abbiamo modificato il file di configurazione di Inetsim con il comando “sudo nano /etc/inetsim/inetsim.conf (Fig. 1)

. Abbiamo deciso di emulare solo i servizi HTTP e HTTPS (Fig. 2) e abbiamo impostato l’indirizzo IP della macchina Kali sulla direttiva `service_bind_address`, decommentando la riga (rimuovendo il carattere #) e assegnando 192.168.50.100 come valore (Fig. 3).

Successivamente abbiamo esaminato i “fake files” messi a disposizione da INetSim, utili per simulare risorse web da visitare durante i test (Fig. 4), e abbiamo salvato il file di configurazione.

A questo punto abbiamo avviato Inetsim con “sudo inetsim” e messo la macchina in ascolto per emulare la navigazione web (Fig. 5). Per verificare il corretto funzionamento abbiamo aperto il browser e visitato prima la pagina principale <http://192.168.50.100> (Fig. 6), quindi alcuni fake file, ad esempio <http://192.168.50.100/sample.txt> (Fig. 7) e <http://192.168.50.100/sample.jpg> (Fig. 8).

Successivamente abbiamo avviato Wireshark e selezionato l’interfaccia loopback (Fig. 9) per catturare i pacchetti generati. Nelle catture abbiamo osservato prevalentemente traffico TCP, con il relativo three-way handshake (SYN / SYN-ACK / ACK) e i segmenti di controllo successivi (ACK, FIN) visibili nelle intestazioni dei pacchetti (Fig. 10).

Inoltre, abbiamo filtrato il traffico HTTP e analizzato richieste e risposte: è possibile visualizzare dettagli come lo User-Agent, la URI completa della richiesta e altri header che possono contenere dati sensibili (Fig. 11).

In conclusione, l’esercitazione è stata completata con esito positivo: la regola firewall su Windows ha permesso le richieste ICMP da Kali e Inetsim ha emulato correttamente il servizio HTTPS, come confermato dalle catture Wireshark.



```
(kali@kali)-[~]  
$ sudo nano /etc/inetsim/inetsim.conf  
[sudo] password for kali:
```

Figura 1 Modifica del file /etc/inetsim/inetsim.conf in nano

```
Session Actions Edit View Help
GNU nano 8.6 /etc/inetsim/inetsim.conf
#####
#
# INetSim configuration file
#
#####

#####
# Main configuration
#####

#####
# start_service
#
# The services to start
#
# Syntax: start_service <service name>
#
# Default: none
#
# Available service names are:
# dns, http, smtp, pop3, tftp, ftp, ntp, time_tcp,
# time_udp, daytime_tcp, daytime_udp, echo_tcp,
# echo_udp, discard_tcp, discard_udp, quotd_tcp,
# quotd_udp, chargen_tcp, chargen_udp, finger,
# ident, syslog, dummy_tcp, dummy_udp, smtps, pop3s,
# ftps, irc, https
#
#start_service dns
#start_service http
#start_service https
#start_service smtp
#start_service smtps
#start_service pop3
#start_service pop3s
#start_service ftp
#start_service ftps
#start_service tftp
#start_service irc
#start_service ntp
#start_service finger
#start_service ident
#start_service syslog
#start_service time_tcp
#start_service time_udp
#start_service daytime_tcp
#start_service daytime_udp
```

Figura 2 Selezione dei servizi in InetSim: abilitazione di HTTP e HTTPS.

```
#####
# service_bind_address
#
# IP address to bind services to
#
# Syntax: service_bind_address <IP address>
#
# Default: 127.0.0.1
#
service_bind_address 192.168.50.100
```

Figura 3 Decommentazione e impostazione di service_bind_address 192.168.50.100


```
#####
# https_fakefile
#
# Fake files returned in fake mode based on the file extension
# in the HTTPS request.
# The fake files must be placed in <data-dir>/http/fakefiles
#
# Syntax: https_fakefile <extension> <filename> <mime-type>
#
# Default: none
#
https_fakefile txt    sample.txt      text/plain
https_fakefile htm    sample.html      text/html
https_fakefile html   sample.html      text/html
https_fakefile php    sample.html      text/html
https_fakefile gif    sample.gif      image/gif
https_fakefile jpg    sample.jpg      image/jpeg
https_fakefile jpeg   sample.jpg      image/jpeg
https_fakefile png    sample.png      image/png
https_fakefile bmp    sample.bmp      image/x-ms-bmp
https_fakefile ico    favicon.ico    image/x-icon
https_fakefile exe    sample_gui.exe  x-msdos-program
https_fakefile com    sample_gui.exe  x-msdos-program
#####
```

Figura 4 Elenco dei “fake files” disponibili in Inetsim (es. sample.txt, sample.jpg).

```
(kali㉿kali)-[~]
└─$ sudo inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
≡ INetSim main process started (PID 3483) ≡
Session ID:      3483
Listening on:    192.168.50.100
Real Date/Time:  2025-10-10 15:36:14
Fake Date/Time:  2025-10-10 15:36:14 (Delta: 0 seconds)
Forking services ...
  * https_443_tcp - started (PID 3486)
  * http_80_tcp  - started (PID 3485)
done.
Simulation running.
█
```

Figura 5 Avvio di Inetsim in console: servizi attivi e porte in ascolto.

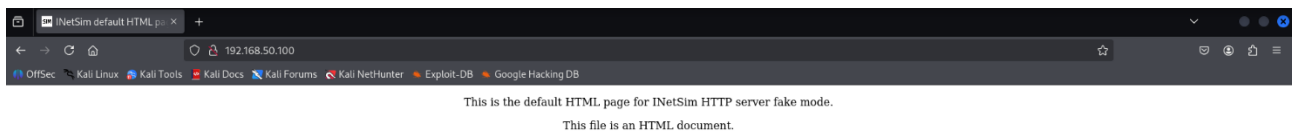


Figura 6 Verifica HTTP: pagina principale servita da Inetsim (<http://192.168.50.100>).

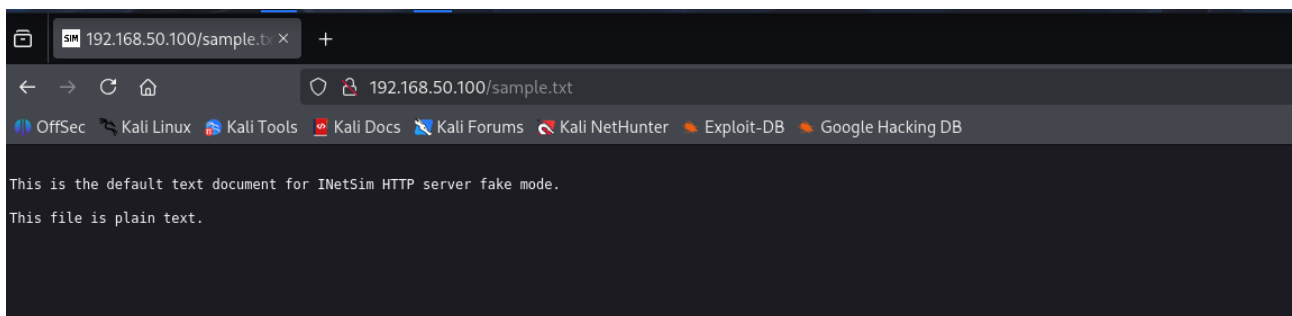


Figura 7 Visualizzazione del fake file sample.txt servito da Inetsim.

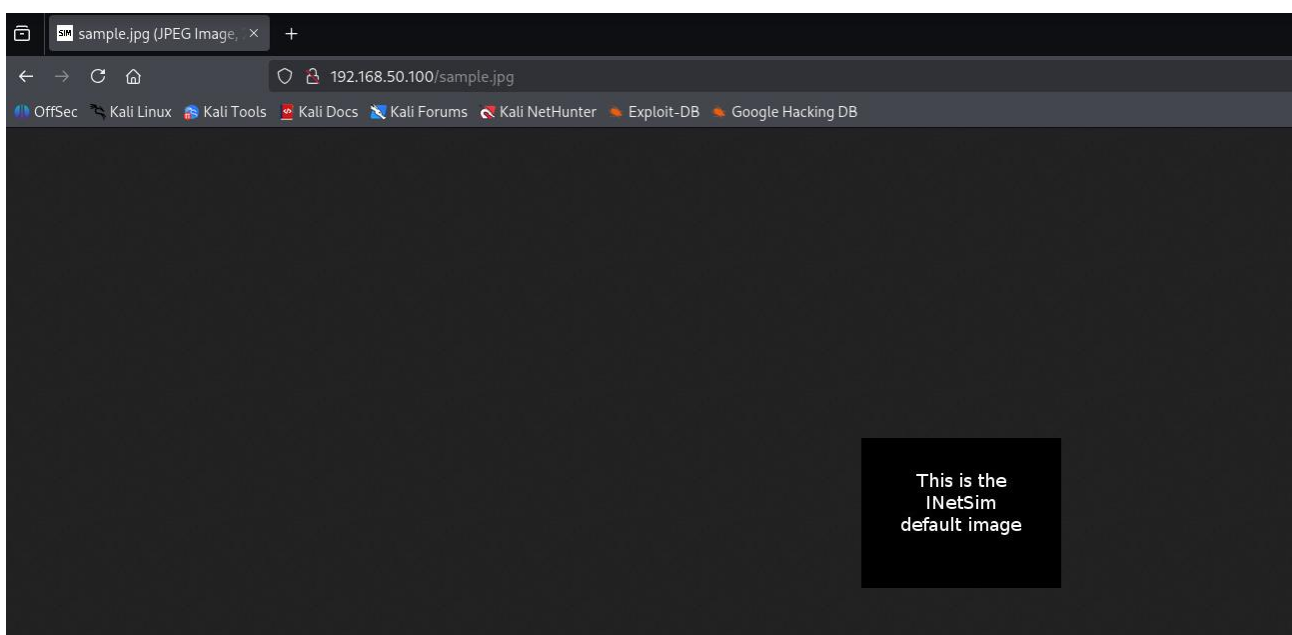


Figura 8 Visualizzazione del fake file sample.jpg servito da Inetsim.

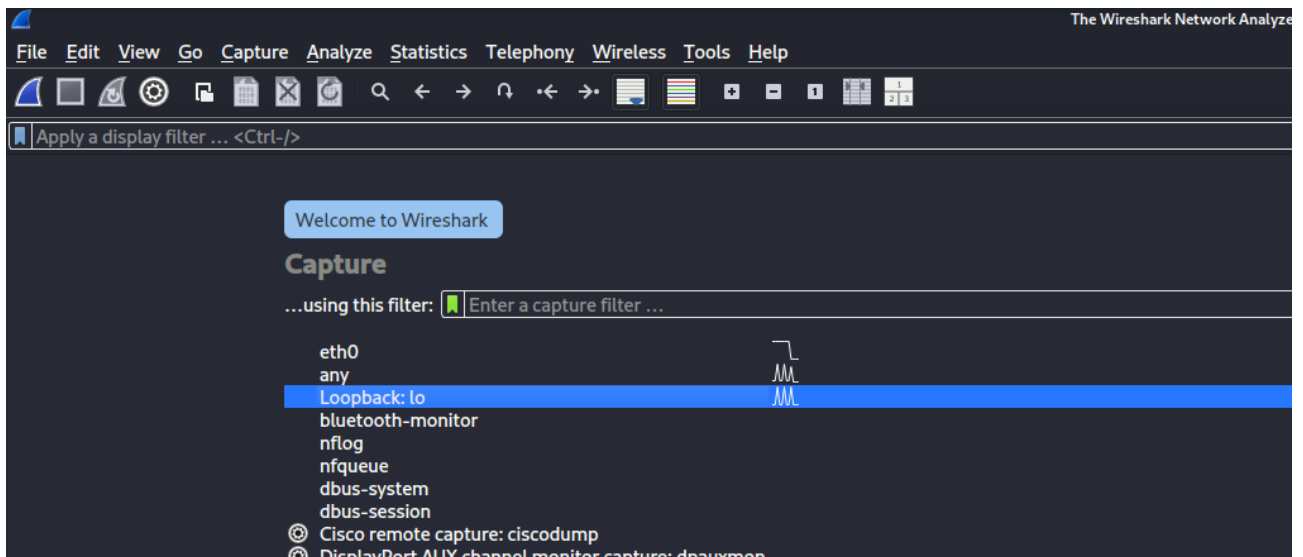


Figura 9 Wireshark: selezione dell'interfaccia di cattura (loopback/lo).

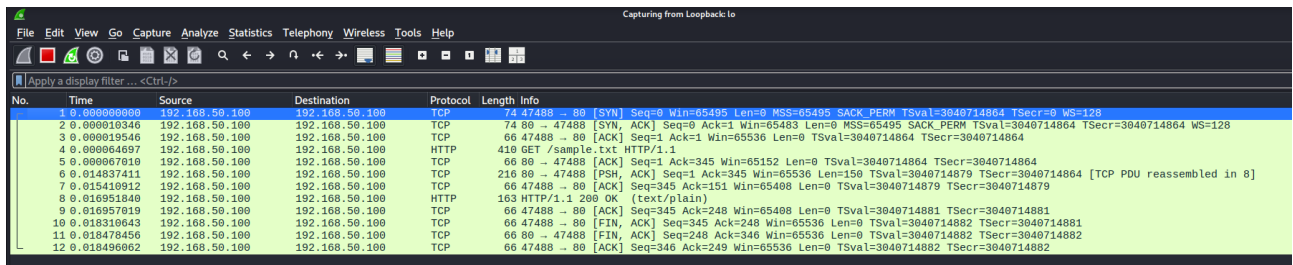


Figura 10 Wireshark: TCP three-way handshake (frame SYN / SYN-ACK / ACK).

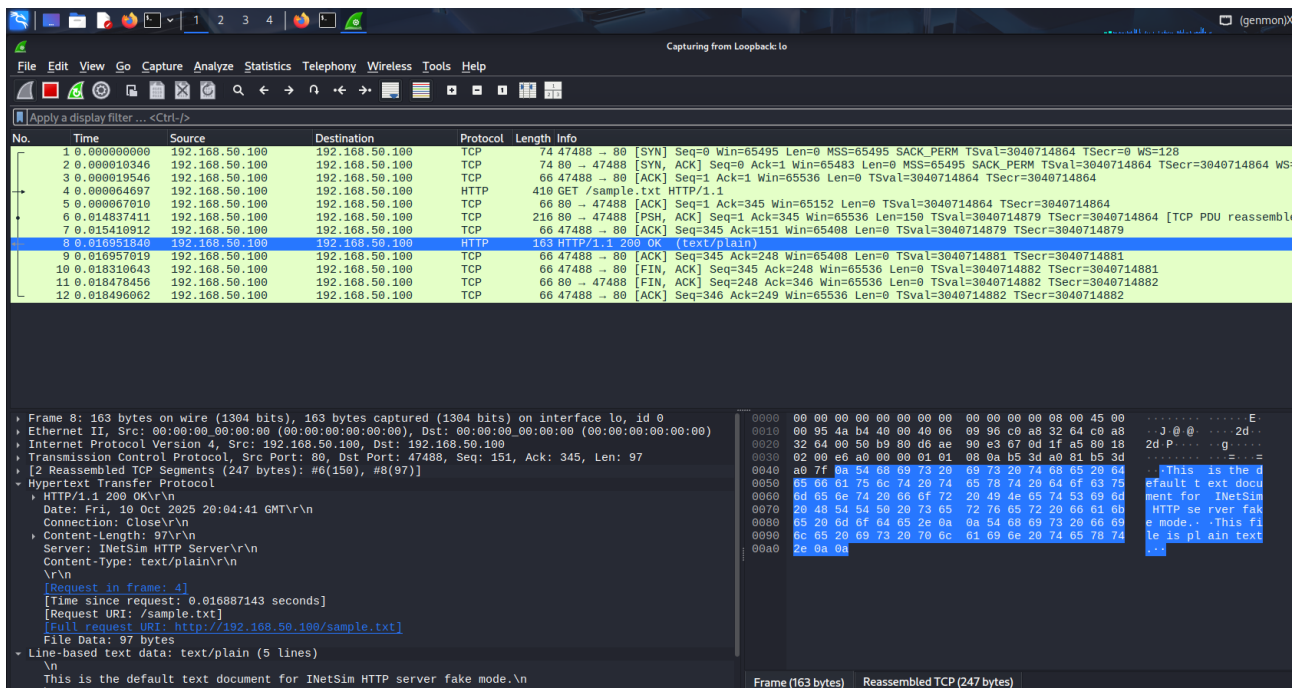


Figura 11 Wireshark: dettaglio di una richiesta HTTP (header visibili: Host, User-Agent, URI).

Venezia, 10/10/2025

Fabio Renna